

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities. (BL5, BL6)

Assessment Tool Weightage: 10%

QUESTIONS: 20

TOTAL MARKS: 20

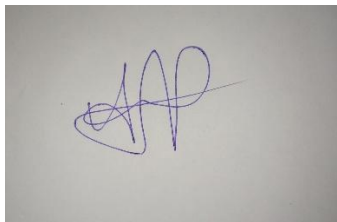
MOCK INTERVIEW

Code of Conduct:

I Mohammed Arshad R\192521126 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a handwritten signature in blue ink on a light-colored surface. The signature is stylized and appears to be 'M. Arshad'.

Mock Interview — Cryptography and Network Security

Q1. Which would you prioritize in a secure network design: performance or security, and how would you balance both?

A: I wouldn't treat this as an either/or choice — security is a non-negotiable baseline, but I'd apply a risk-based design so performance doesn't become collateral damage. Critical assets get the strongest controls even at some performance cost, while lower-risk traffic gets lighter-weight controls. Practically, I'd offload expensive operations like encryption to dedicated hardware, cache decisions for already-validated sessions, and push non-time-critical deep inspection to an out-of-band analysis path rather than the inline fast path. That way the network stays fast for the vast majority of legitimate traffic while the sensitive paths still get full scrutiny.

Q2. Can you explain the difference between stateful and stateless firewalls, and which one is more suitable for enterprise networks?

A: A stateless firewall evaluates every packet in isolation against static rules, with no memory of prior traffic — it's fast and simple, but it has to open broad rule ranges just to allow return traffic, and it can't tell a spoofed packet from a genuinely established connection. A stateful firewall maintains a connection table tracking each session's state, so it automatically permits return traffic that belongs to a session it actually saw get established, and rejects anything that doesn't match a tracked session. For enterprise networks, I'd recommend stateful — modern hardware easily absorbs the extra overhead, and the security benefit of connection-aware filtering is significant given the scale and sensitivity of enterprise traffic.

Q3. If an IDS generates too many false positives, how would you optimize its performance?

A: First, I'd audit the current rule set and remove or narrow overly broad signatures that are triggering on benign traffic. For anomaly-based components, I'd re-baseline what 'normal' looks like for this specific network, since a generic baseline is a common cause of false alarms. I'd also whitelist known-good hosts and behaviors, and require correlation across multiple weak signals before raising a high-confidence alert instead of firing on any single indicator. Finally, I'd build a feedback loop so analyst-confirmed false positives feed back into tuning the rules, so the system actually gets better over time rather than repeating the same false alarms.

Q4. How would you design a real-time intrusion detection system for high-speed networks?

A: I'd use a tiered fast-path/slow-path architecture. The fast path does signature matching against known threats using hash-indexed or Aho-Corasick-style multi-pattern search, which can run at close to line rate with hardware acceleration or SmartNIC/DPDK offload. The slow path only analyzes a sampled subset of flows with heavier anomaly or behavioral techniques, so the expensive analysis never becomes the bottleneck for the full traffic volume. I'd also scale horizontally with multiple parallel sensors rather than relying on a single inspection point, since that's usually the only way to sustain both high throughput and low latency simultaneously.

Q5. What are the key challenges in detecting zero-day malware, and how would you address them?

A: The core challenge is that, by definition, no signature exists yet, so detection has to rely on behavior rather than known patterns — and behavioral detection naturally has a higher false-positive rate and needs more computational effort. Sandboxing helps, but sophisticated malware can detect it's in a sandbox and simply stay dormant. I'd address this with layered behavioral analytics — endpoint detection and response (EDR) watching for suspicious process behavior, machine-learning models trained on behavior patterns rather than static signatures, and fast threat-intelligence sharing so that once any organization identifies a new threat, the detection window for everyone else shrinks quickly.

Q6. How would you differentiate between signature-based and anomaly-based IDS, and when would you use each?

A: Signature-based IDS matches traffic against a database of known attack patterns — it's fast and has a low false-positive rate, but it's blind to anything not already in that database. Anomaly-based IDS builds a baseline of normal behavior and flags deviations from it, which lets it catch novel or zero-day attacks, but at the cost of a higher false-positive rate and the need for careful baseline tuning. I'd use signature-based detection as the fast, high-confidence layer for well-known threats, and layer anomaly-based detection on top specifically to catch what signatures can't — in practice, I'd run both together rather than choosing one.

Q7. If a network is under a DDoS attack, what immediate and long-term mitigation strategies would you apply?

A: Immediately, I'd engage upstream mitigation — either an ISP-level scrubbing service or a CDN/cloud-based DDoS protection provider — and apply rate limiting or blackholing against clearly malicious source traffic to relieve pressure on the network. If the infrastructure supports it, I'd also scale up capacity temporarily to absorb the spike. Longer term, I'd invest in redundant, geographically distributed infrastructure with anycast routing, keep always-on DDoS protection active rather than reactive, improve traffic baselining so attacks are caught earlier, and make sure there's a rehearsed incident-response runbook specifically for DDoS scenarios so the team isn't improvising during a live attack.

Q8. How would you secure remote access to an enterprise network?

A: I'd start with a strong VPN using modern protocols like WireGuard or IPsec, but I wouldn't stop there — I'd enforce multi-factor authentication on every remote login, and apply least-privilege, role-based access so a remote connection only reaches the specific resources that user actually needs, not the whole internal network. I'd also check device posture/compliance before granting access, since a compromised or unpatched device is a common entry point. If I were designing this today, I'd seriously consider a Zero Trust Network Access model instead of a traditional perimeter VPN, since it verifies every request continuously rather than trusting a device just because it's connected.

Q9. What are the limitations of traditional antivirus software, and how can modern solutions overcome them?

A: Traditional antivirus relies heavily on signature matching, so it's essentially blind to zero-day malware, polymorphic variants, and fileless attacks that abuse legitimate system tools rather than dropping a malicious file to scan. It's also often resource-heavy during full-system scans. Modern endpoint detection and response (EDR/XDR) solutions address this by monitoring process behavior and system activity in real time instead of relying solely on static signatures, correlating telemetry across many endpoints to spot patterns a single machine wouldn't reveal, and pulling continuously updated threat intelligence from the cloud rather than periodic signature downloads.

Q10. How would you design a secure email filtering system to prevent phishing and spam attacks?

A: I'd build it in layers. First, sender authentication using SPF, DKIM, and DMARC to catch domain spoofing before content is even examined. Then content-level analysis — scanning embedded links against threat-intelligence blocklists, sandboxing attachments before delivery, and using a machine-learning classifier trained on labeled spam and phishing examples for the harder cases. I'd also analyze headers for red flags like mismatched reply-to addresses or urgency language. Finally, I'd build in a user-reporting feedback loop so confirmed phishing reports continuously improve the filter, and pair the technical layer with ongoing user security-awareness training, since no filter catches everything.

Q11. If an attacker bypasses firewall rules, what additional layers of security would you implement?

A: This is exactly why I design for defense-in-depth rather than relying on the firewall as the single control. Behind it, I'd want IDS/IPS to catch what slipped through, network segmentation so a breach in one zone can't freely reach others, and endpoint detection and response on individual hosts to catch malicious activity even after the network boundary is crossed. I'd also make sure SIEM correlation and continuous monitoring are in place to detect the post-bypass activity quickly, and I'd run a focused penetration test to understand exactly how the bypass happened so I can close that specific gap, not just add more generic controls.

Q12. *How would you evaluate the effectiveness of network segmentation in preventing cyberattacks?*

A: I wouldn't just look at the architecture diagram — I'd test it. That means running penetration tests or red-team exercises that specifically try to move laterally from a compromised segment into others, and reviewing the actual firewall/ACL rules between segments for overly permissive entries that undermine the intended isolation. I'd also monitor cross-segment traffic logs for flows that shouldn't be happening, and, over time, track how well segmentation actually contained the blast radius during real incidents. Ultimately, segmentation is proven by containment outcomes, not by how good the network diagram looks.

Q13. *What role does encryption play in preventing malware attacks, if any?*

A: Honestly, encryption's role here is limited and mostly indirect — it doesn't stop malware from executing. Where it helps is protecting data confidentiality, so if malware does exfiltrate data, that data may be far less useful to the attacker without the keys. Code signing, which relies on cryptographic verification, does help prevent unauthorized or tampered software from running. But encryption also cuts both ways from a defensive standpoint: encrypted channels can hide malware in transit from inspection, which is exactly why organizations need TLS inspection at the perimeter if they want visibility into encrypted traffic without giving up the ability to detect threats hidden inside it.

Q14. *How would you secure a cloud-based network infrastructure?*

A: I'd start with strong identity and access management — least privilege, MFA, and tightly scoped roles — since misconfigured access is one of the most common causes of cloud breaches. I'd use virtual network segmentation through VPCs, subnets, and security groups, encrypt data both at rest and in transit, and make sure logging and monitoring are continuous and integrated with a SIEM. I'd also be very deliberate about the shared-responsibility model, so it's clear exactly what the cloud provider secures versus what my team is responsible for, and I'd scan infrastructure-as-code templates for misconfigurations before they ever reach production.

Q15. *What is the importance of log analysis and monitoring in network security?*

A: Logs are the evidence trail that makes detection, investigation, and response possible at all. Without centralized log analysis, a slow, multi-stage attack — like what happened in the TJX breach, where attackers had roughly 18 months of undetected access — can persist far longer than it should. Continuous monitoring turns that into real-time alerting instead of after-the-fact discovery, supports compliance and audit requirements, and when logs from many sources are correlated together in a SIEM, patterns become visible that no single log source would reveal on its own.

Q16. *How would you respond to a security breach in a corporate network?*

A: I'd follow a structured incident-response lifecycle: first identify and scope the breach to understand what's actually been affected, then contain it by isolating the affected systems or segments to stop further spread. Next, eradicate the root cause — removing the malware and closing whatever vulnerability was exploited — before recovering by restoring systems from verified clean backups. Throughout, I'd preserve forensic evidence and keep clear communication going with stakeholders and, where required, regulators or affected customers. Afterward, I'd run a post-incident review so the lessons actually get built back into our defenses, not just documented and forgotten.

Q17. What are the advantages and limitations of next-generation firewalls (NGFW)?

A: NGFWs combine traditional packet filtering with deep packet inspection, application-level awareness, integrated intrusion prevention, and often identity-based policies and SSL/TLS inspection — giving far richer visibility and control than a simple port-and-IP firewall. The trade-offs are real, though: that extra inspection is computationally expensive and can affect throughput and latency, the configuration is more complex to get right, and if TLS inspection isn't properly enabled, fully encrypted or cleverly obfuscated traffic can still slip past the deeper inspection capabilities that make an NGFW valuable in the first place.

Q18. How would you integrate IDS/IPS with SIEM systems for better threat detection?

A: I'd forward all IDS/IPS alerts and logs into the SIEM so they can be correlated with logs from firewalls, endpoints, servers, and authentication systems rather than being reviewed in isolation. That correlation is where the real value is — a single IDS alert might be low-confidence on its own, but if the SIEM connects it to an anomalous login that happened right afterward, that combination becomes a high-confidence signal. I'd also look at integrating SOAR playbooks so that when the SIEM confirms a genuinely correlated threat, containment actions can trigger automatically instead of waiting on a manual response.

Q19. What are the best practices for patch management and vulnerability assessment?

A: It starts with an accurate, current asset inventory — you can't patch what you don't know you have. From there, I'd run regular automated vulnerability scans and prioritize patches by actual risk: severity, real-world exploitability, and how critical the affected asset is, not just a raw CVSS score. I'd always test patches in a staging environment before pushing to production, maintain a clear SLA for how quickly different severity levels must be patched, and track compliance metrics over time. I'd also complement scanning with periodic penetration testing, since scanning alone tends to miss logic flaws and chained vulnerabilities.

Q20. How would you design a future-proof network security architecture considering emerging threats?

A: I'd move away from a traditional perimeter-trust model toward Zero Trust — verifying every user, device, and request continuously, regardless of where it's coming from. I'd build in defense-in-depth so there's no single point of failure, and design for automation from the start, since manual processes simply can't keep pace with how fast the threat landscape evolves. I'd incorporate AI/ML-based detection so the system can adapt to new attack patterns rather than only recognizing what it was explicitly programmed to catch, treat identity as the real perimeter, and build in continuous monitoring and regular architecture reviews so the design keeps evolving instead of becoming another TJX-style cautionary tale a few years down the line.